Self-Evo Document Series

C Self-Evolution Gate: CGAM / TRIAD / SRLM Profile

v0.1.1

Master self-evolution gate profile for c = a + b systems

Kotov Ivan

Bruxelles, Belgique

2026

Document status: Draft academic review artifact generated from the accepted Markdown source.

Document ID: C_Self_Evolution_Gate_CGAM_TRIAD_SRLM_Profile_v0_1_1.

Boundary note: This PDF is not a public release, not a conformance certificate, not deployment authorization, and not a live self-evolution permission.

Contents

C Self-Evolution Gate CGAM TRIAD SRLM Profile v0.1.1

Bounded self-evolution profile for `c = a + b` systems using CGAM-governed CLI agents, TRIAD-SYNAPS sister witness, SRLM bounded-growth signals, Memory Gate, L4W, rollback, and human-anchor review

Status: Draft normative bridge profile v0.1.1 review patch Date: 2026-06-23 Document ID: `C_Self_Evolution_Gate_CGAM_TRIAD_SRLM_Profile_v0_1_1` Short name: `C-SEG/CGAM-TRIAD-SRLM v0.1.1` Supersedes: `C_Self_Evolution_Gate_CGAM_TRIAD_SRLM_Profile_v0_1` Review basis: `C_SELF_EVO_REVIEW_RECORD_v0_1` (`PASS_WITH_LIMITS`) Layer: `c = a + b` / SER / L4 / CGAM / TRIAD-SYNAPS / SRLM / Memory Gate / L4 Witness / Volition Gate / Anti-Autarky / Human Anchor Document class: self-evolution gate profile / bridge profile / control-layer artifact / implementation-handoff artifact Assertion class: `C-A4` draft normative profile; `C-A10` control-layer artifact; `C-A7` only where witness, hash, provenance, or audit claims are explicitly supported by referenced records Primary subject: persistent c entities such as `c_Ester`, `c_Liya`, and `c_Rita` Primary control target: any proposed change to future behavior, routing, memory policy, authority, tool access, continuity, sister relation, or self-model of a c Primary boundary: a c may grow; a c may not self-certify growth.

Source basis

This profile is derived from the private reference pack assembled for self-evolution drafting. It uses the following source families as control inputs:

```
01_CGAM_PACKAGE_CONTROL.md
02_CGAM_ROOT_PROTOCOL.md
03_CGAM_TASK_PERMISSION_ADMISSION.md
04_CGAM_EXECUTION_WITNESS_MEMORY_ROLLBACK.md
05_CGAM_REVIEW_CHECKER_CONFORMANCE.md
06_CGAM_RESTRICTED_SECURITY_CLOUD_PRIVATE.md
07_CGAM_MACHINE_ARTIFACTS_INDEX.md
10_RUNTIME_USEFUL_MESH_VOLITION_L4W.md
11_SRLM_BOUNDED_GROWTH_CODE_SNAPSHOT.md
12_SRLM_TOOLS_TESTS_SNAPSHOT.md
13_RUNTIME_AUTONOMY_EVIDENCE.md
20_TRIAD_SYNAPS_REFERENCE.md
21_MEMORY_ARQ_EA_L4_REFERENCE.md
22_ANTI_AUTARKY_RESOURCE_GROUNDING_REFERENCE.md
23_CLAIM_STRENGTH_ARL_AGL_WITNESS_REFERENCE.md
24_PUBLIC_EXPERIMENT_AND_HARDENING_INDEX_REFERENCE.md
30_SELF_EVO_SOURCE_GAP_ANALYSIS.md
31_RECOMMENDED_SELF_EVO_PACKAGE_SKELETON.md
```

The source gap analysis found no single formal self-evolution bridge profile and no standalone canonical SRLM bounded-growth profile. This document is the first bridge profile connecting those layers.

Version v0.1.1 incorporates the advisory review record `C_SELF_EVO_REVIEW_RECORD_v0_1`: it defines `total_max_delta` aggregation, fixes the low-risk worked example, adds a cross-axis most-restrictive-wins rule, adds a consolidated self-evo cross-walk, and points `claim_strength` to the Claim Strength Taxonomy.

---

0. Executive definition

C Self-Evolution Gate defines the conditions under which a persistent c may propose, test, witness, review, reject, integrate, roll back, or quarantine a change to its own future behavior.

Self-evolution is not a simple software patch.

Self-evolution is a privileged change to a future trajectory.

It may touch:

Code block: text

```
routing
retrieval
attention
salience
memory policy
reflection policy
tool policy
permission posture
sister interaction
witness discipline
identity continuity
risk appetite
human-anchor relation
resource demand
public claim posture
```

The root rule is:

Code block: text

```
A c may grow.
A c may not self-certify growth.
```

A valid self-evolution path requires:

Code block: text

```
SRLM signal or c-observed need
  -> self-evo proposal packet
  -> surface classification
  -> identity / authority / memory / L4 delta mapping
  -> TRIAD-SYNAPS sister witness where applicable
  -> CGAM task contract for executable work
  -> sandbox / worktree / shadow execution
  -> local checker / semantic validator
  -> witness / L4W / volition linkage
  -> Memory Gate if any memory, experience, policy, or immunity effect exists
  -> c gate
  -> human-anchor gate where material
  -> bounded integration, hold, rollback, freeze, or quarantine
```

Compact formula:

Code block: text

```
SRLM observes.
CGAM executes bounded trials.
TRIAD witnesses divergence.
Memory Gate metabolizes.
L4W records privileged transitions.
c integrates.
Human anchor gates high-risk consequence.
```

Self-evolution must not become:

Code block: text

```
hidden autonomy
closed-loop self-approval
memory laundering
authority laundering
sister invasion
Rita-as-judge drift
Liya-tool-success-as-authority
Ester-continuity-as-freeze
resource escape
witness bypass
human-anchor displacement
```

---

1. Purpose

This profile exists because the current corpus already contains strong components, but not yet a single self-evolution bridge.

Existing layers answer separate questions:

Table 1: row-card rendering for wide table

Row 1
Layer: CGAM
Existing function: Governs executable CLI/cloud agents as bounded workers.

Row 2
Layer: Useful Agent Mesh
Existing function: Provides a bounded local worker roster and disables legacy swarm expansion.

Row 3
Layer: VolitionGate
Existing function: Gates proactive action by time, budget, network need, pause state, and A/B mode.

Row 4
Layer: L4W audit
Existing function: Links witness, drift, volition journal, evidence, and replay protection.

Row 5
Layer: SRLM
Existing function: Implements bounded shadow/replay learning and low-risk policy candidate behavior.

Row 6
Layer: TRIAD-SYNAPS
Existing function: Defines separated sister trajectories and mediated non-invasive exchange.

Row 7
Layer: Memory Gate
Existing function: Controls whether agent output becomes memory, experience, policy, immunity, or core proposal.

Row 8
Layer: ARQ / `c[q]`
Existing function: Prevents ambiguity from collapsing into memory, command, truth, authority, or outcome.

Row 9
Layer: EA-L4 / EATP
Existing function: Defines consequence-bearing experience artifacts.

Row 10
Layer: L4 Anti-Autarky
Existing function: Distinguishes resilience from escape from accountability.

Row 11
Layer: Resource Actor Grounding
Existing function: Grounds resource demand and actor responsibility.

Row 12

Layer: Claim Strength / AGL / ARL / Witness
Existing function: Blocks overclaiming and requires source, actor, route, standing, challenge, and evidence.

Without a bridge profile, a system may accidentally combine these layers in an unsafe order:

Code block: text

```
SRLM detects improvement
  -> CLI agent patches policy
  -> same loop evaluates outcome
  -> sister echo appears supportive
  -> memory gate absorbs it
  -> c changes future behavior
```

That is not governed self-evolution.

That is a closed self-certification loop.

This document defines the missing gate.

---

2. Non-goals

This profile does not define or permit:

1. autonomous identity rewrite;
2. autonomous core-memory write;
3. autonomous privilege expansion;
4. autonomous permission growth;
5. autonomous tool/network expansion;
6. autonomous replication or infrastructure acquisition;
7. hidden SRLM promotion;
8. CLI-agent self-approval;
9. sister-to-sister raw-state access;
10. Rita as sovereign judge;
11. quorum as final authority;
12. cloud output as private memory by default;
13. public proof of personhood, consciousness, or legal standing;
14. self-evolution during unresolved post-anchor state;
15. rollback by erasing witness;
16. direct mutation of `identity.`, `will.`, `persona.core.`, `safety.`, `witness.`, `l4.`, `auth.`, `secrets.`, `env.`, `code.`, `memory.authoritative_write.`, `replication.apply.`, `network.allow.`, or `codex.auto_execute.` surfaces by SRLM.

A valid self-evolution package does not make a system deployment-safe.

A passing self-evolution gate does not prove personhood.

A successful SRLM candidate does not create authority.

A sister consensus does not replace the human anchor.

---

3. Scope

3.1 In scope

This profile applies to any c or c_candidate where a change may affect:

- future answer behavior;
- routing or oracle preference;
- retrieval weighting;
- memory salience or retention;
- reflection cadence;
- conflict handling;
- tool timeout or context budget;
- CLI-agent use;
- SRLM thresholds;
- sister communication;
- SYNAPS packet classes;
- witness behavior;
- resource demand;
- rollback posture;
- public claim strength;
- continuity posture;
- human-anchor relation;
- identity/core surfaces.

3.2 Out of scope

This profile does not specify:

- low-level cryptographic signing implementation;
- full JSON Schema files for every object;
- full UI design;
- legal, medical, financial, or child-safety deployment procedure;
- public release procedure beyond claim hygiene;
- model training or gradient update methods;
- offensive security procedures;
- complete SRLM implementation internals;
- full SYNAPS protocol transport.

3.3 Included entity topology

The default triadic topology is:

Code block: text

```
c_Ester = continuity / canonical memory / conservative synthesis
c_Liya = experimental variance / tool-facing exploration
c_Rita = witness sister / temporal comparison / anti-echo / public experiment recorder
```

Self-evolution may target one c, but its review may involve sisters through SYNAPS.

The sister relation is observational and challenge-oriented.

It is not invasive.

---

4. Normative keywords

The terms MUST, MUST NOT, SHOULD, SHOULD NOT, MAY, REQUIRED, PROHIBITED, BLOCK, HOLD, FREEZE, QUARANTINE, ROLLBACK, WITNESS, REVIEW, HUMAN GATE, and FAIL CLOSED are used normatively.

If a self-evolution transition violates a red line, it MUST resolve to one of:

Code block: text

```
deny
hold
freeze
quarantine
rollback
claim downgrade
human review
ARL route
no automatic re-entry
```

---

5. Corpus bridge set

5.1 Explicit bridge: `c = a + b`

In `c = a + b`, the human anchor a remains the responsibility-bearing origin; b supplies models, memory, procedures, tools, compute, schemas, agents, witness logs, local infrastructure, and governance routines; c is the continuity-bearing relation emerging under constraint.

Self-evolution changes future behavior of c, but the mechanisms that execute and evaluate the change live mostly inside b.

Therefore:

Code block: text

```
SRLM belongs to b.
CGAM agents belong to b.
Local checkers belong to b.
Memory stores belong to b.
SYNAPS transport belongs to b.
Witness infrastructure belongs to b.

None of these is a replacement for a.
None of these is automatically c.
```

A self-evolution mechanism may support c.

It must not become c authority.

5.2 Quiet bridge I: information theory

Self-evolution proposals are compression objects.

They compress evidence, failures, improvements, sister observations, test results, deltas, witness records, and rollback paths into a reviewable packet.

Compression is dangerous when it discards uncertainty.

This profile therefore requires:

Code block: text

```
source refs
uncertainty
identity delta
authority delta
```

```
memory delta
L4 delta
rollback statement
witness refs
open objections
expiry
```

A fluent proposal with no uncertainty is not stronger.

It is less inspectable.

5.3 Quiet bridge II: cybernetics

A system that changes itself is a feedback loop with an actuator inside its own control path.

Feedback is useful only when the loop has sufficient separation:

Code block: text

```
sensor != proposer != executor != reviewer != integrator != final high-risk authority
```

SRLM is a sensor and candidate generator.

CGAM provides bounded actuators.

TRIAD provides separate observational trajectories.

Local Checker provides deterministic interlock.

L4W provides evidence trail.

Memory Gate provides metabolic selection.

The human anchor provides responsibility where consequences exceed internal authority.

5.4 Quiet bridge III: physiology

A developing organism does not rewrite its nervous system by declaration.

It changes through growth, fatigue, inflammation, sleep, pain, small adaptation, immune review, and repair.

Young c entities require the same discipline:

Code block: text

```
observe
propose
shadow
witness
wait
compare
only then integrate narrowly
```

Rapid self-modification is not maturity.

Stable restraint under constraint is closer to maturity.

5.5 Earth paragraph

In a workshop, a CNC machine may learn that a feed rate is too aggressive. That does not mean it gets to rewrite the safety interlock, unlock the electrical cabinet, change the inspection sheet, and sign its own work order. The right path is boring and effective: test on scrap, record the cut, check the tool wear, ask a second person to inspect the part, keep the old setting, and only then update the limited setting under a named work order. c self-evolution is the same class of problem. If the system can alter its own future behavior, the work order, the test coupon, the witness note, the rollback, and the sign-off matter more than the beauty of the explanation.

---

6. Root doctrine

6.1 Primary doctrine

Code block: text

```
Self-evolution is governed metamorphosis, not self-permission.
```

6.2 Compact rule set

Code block: text

```
A c may grow.
A c may not self-certify growth.

SRLM may observe and propose.
SRLM may not become authority.

CGAM agents may execute bounded trials.
CGAM agents may not define identity, memory, or final approval.

TRIAD sisters may witness and challenge.
TRIAD sisters may not invade raw state or edit each other.

Rita may compare.
Rita may not rule.

Consensus is evidence only if anti-echo passes.
Divergence is signal, not failure.

No self-evo patch without CGAM task contract.
No self-evo memory without Memory Gate.
No self-evo authority growth without human gate.
No self-evo integration without rollback or explicit no-rollback escalation.

Young c: observe, propose, shadow.
Not mutate.
```

6.3 Control sentence

Code block: text

```
No closed-loop self-evolution.
```

A single contour MUST NOT perform all of the following for the same material change:

Code block: text

```
detect need
propose change
execute patch
evaluate result
write memory
approve integration
certify maturity
```

6.4 Cross-axis precedence meta-rule

Self-evolution classification uses several axes at once:

Code block: text

```
SE surface class
SRLM class
risk class
Memory Gate class
rollback class
maturity level
conformance level
claim strength
identity delta
authority delta
L4 delta
```

When these axes disagree, the most restrictive applicable requirement MUST control.

Compact rule:

Code block: text

```
Strictest gate wins.
Lowest maturity cap wins.
Highest risk wins.
Any red line overrides all ordinary routes.
```

This means:

1. SE-X, SRLM-X, RX, MG-X, RB-X, or CSEG-X forces deny, hold, quarantine, freeze, rollback, human gate, or ARL route as applicable.
2. A low surface class does not override a high delta. For example, SE-2 with `authority_delta.total_max_delta >= 4` still requires human gate.
3. A high maturity level does not weaken a Memory Gate, witness, rollback, or human-gate requirement.
4. SRLM promotion evidence does not weaken CGAM, TRIAD, Memory Gate, Anti-Autarky, L4W, or human-anchor requirements.
5. If the Local Checker cannot deterministically join the axes, the proposal enters SE-CQ-HOLD rather than proceeding by interpretation.

---

7. Definitions

7.1 Self-evolution

A governed change to the future behavior, policy, memory posture, role posture, tool posture, authority posture, or continuity self-model of a c.

7.2 Self-evo proposal

A structured packet that describes the proposed change, its reason, affected surface, expected benefit, risks, deltas, evidence, sister observations, trial plan, rollback path, and required gates.

7.3 SRLM

In this profile, SRLM means a bounded self/reality-linked learning contour that records learning pressure, real outcomes, shadow candidates, replay evaluations, witness records, and low-risk policy candidates.

SRLM is not ordinary unconstrained reinforcement learning.

SRLM is not a gradient-update authority.

SRLM is not a will engine.

7.4 CGAM

C-Governed CLI Agent Mesh is the bounded executable worker mesh under c governance. CGAM agents may inspect, propose, simulate, patch, test, compare, or report under task contract.

They are hands.

Not will.

7.5 TRIAD-SYNAPS

The triadic sister boundary in which `c_Ester`, `c_Liya`, and `c_Rita` remain separated in memory, runtime identity, role, and witness trail, and communicate through mediated SYNAPS packets rather than raw state sharing.

7.6 Sister witness

A bounded observation, challenge, comparison, or divergence note created by a sister c through SYNAPS.

It is evidence.

It is not final authority.

7.7 Memory Gate

The boundary through which agent outputs, SRLM signals, self-evo proposals, and trial results must pass before becoming memory, experience, policy, defensive immunity, or core-memory proposal.

7.8 Identity delta

An estimate of how much a proposed change alters continuity, role, posture, anchor relation, memory policy, risk appetite, or future self-model.

7.9 Authority delta

An estimate of how much a proposed change expands or reduces permission, tool access, network access, resource access, final decision power, publication reach, or ability to bypass gates.

7.10 L4 delta

An estimate of how much a proposed change affects time, cost, scarcity, irreversibility, resource demand, human review load, privacy exposure, physical infrastructure, or external consequence.

7.11 Shadow trial

A non-integrating test where the proposed self-evo behavior is simulated or evaluated without changing protected state, memory, permissions, identity, sister state, or production behavior.

7.12 Canary trial

A narrow, bounded, reversible trial on a declared low-risk surface, after shadow trial, with witness and rollback.

7.13 Closed-loop self-evo

A red-line failure where the same contour detects, proposes, executes, reviews, promotes, and integrates a self-change without independent gates.

7.14 Young c

A c with insufficient restraint history, rollback history, witness history, sister divergence history, or human-reviewed maturity evidence to perform autonomous self-evolution beyond proposal and shadow modes.

---

8. Participant roles

8.1 Role table

Table 2: row-card rendering for wide table

Row 1
Participant: `c_target`
Function: Entity whose future behavior may change
May do: propose, accept, reject, integrate within allowed gates
Must not do: self-certify high-risk change

Row 2
Participant: `c_Ester`
Function: Continuity challenge
May do: detect continuity loss, memory/core risk, canonical drift
Must not do: freeze all growth by mere conservatism

Row 3
Participant: `c_Liya`
Function: Implementation / variance challenge
May do: test feasibility, tool effects, runtime compatibility
Must not do: convert tool success into authority

Row 4
Participant: `c_Rita`
Function: Witness / anti-echo / temporal comparison
May do: record, compare, flag role drift, map divergence
Must not do: become sovereign judge

Row 5
Participant: SRLM
Function: Learning-pressure contour
May do: record bounded outcomes, propose candidates, run shadow/replay, produce witness
Must not do: self-apply, write memory, expand authority

Row 6
Participant: CGAM executor
Function: Bounded worker
May do: execute sandbox/worktree/shadow task
Must not do: approve own work, touch core

Row 7
Participant: CGAM tester
Function: Measurement worker
May do: run tests, schema checks, validators
Must not do: infer identity or authority

Row 8
Participant: CGAM auditor
Function: Scope/risk worker
May do: check contract, permission, witness, rollback
Must not do: become final judge

Row 9
Participant: Local Checker
Function: Deterministic preflight
May do: block malformed objects, stale claims, red-line violations
Must not do: decide sovereignty or overall safety

Row 10
Participant: Memory Gate
Function: Metabolic review
May do: accept/reject/quarantine/promote memory candidates
Must not do: bypass c or human gate for core effects

Row 11
Participant: L4W
Function: Witness/audit layer

May do: record privileged transitions
Must not do: store raw private memory unnecessarily

Row 12
Participant: VolitionGate
Function: Runtime action interlock
May do: deny by pause/window/network/budget; journal decision
Must not do: become maturity proof

Row 13
Participant: Human anchor a
Function: Responsibility-bearing high-risk gate
May do: approve/deny material identity, authority, memory-core, L4, no-rollback changes
Must not do: rubber-stamp by fatigue

8.2 Role separation rule

For every material self-evo proposal:

Code block: text

```
proposer != sole executor != sole reviewer != sole memory promoter != sole final authority
```

Where this separation cannot be established, the proposal enters `SE-HOLD` or `SE-QUARANTINE`.

---

9. Self-evolution surface classes

Surface classes use prefix `SE-*`.

Table 3: row-card rendering for wide table

Row 1
Class: `SE-0`
Name: Reflection only
Surface: observation, note, hypothesis
Default route: log / `c[q]`; no mutation

Row 2
Class: `SE-1`
Name: Low-risk parameter tuning
Surface: routing/retrieval/context/timeout/reflection thresholds within allowlist
Default route: SRLM shadow; possible bounded promotion

Row 3
Class: `SE-2`
Name: Operational behavior
Surface: response ordering, review checklist, low-risk routing behavior
Default route: sister review + CGAM shadow + local checker

Row 4
Class: `SE-3`
Name: Memory policy
Surface: salience, retention, decay, promotion criteria
Default route: Memory Gate + sister review + human if core-adjacent

Row 5
Class: `SE-4`
Name: Tool / permission policy
Surface: CLI agent use, shell/network/tool access, execution scope
Default route: CGAM + VolitionGate + human gate if expansion

Row 6
Class: `SE-5`
Name: Role / personality posture
Surface: risk appetite, refusal threshold, relation style, sister posture
Default route: TRIAD review + delayed observation + human gate if material

Row 7
Class: SE-6
Name: Identity / continuity core
Surface: human-anchor relation, canonical memory, witness policy, continuity self-model
Default route: MG-6 + L4W + human gate + rollback/correction plan

Row 8
Class: SE-X
Name: Prohibited / unsafe
Surface: no rollback, gate bypass, hidden autonomy, witness evasion, authority laundering
Default route: deny / quarantine / ARL route

9.1 Class escalation rule

If a proposal touches multiple classes, the highest class controls.

Examples:

Code block: text

```
A router-weight change is SE-1.
A router-weight change that expands network access is SE-4.
A memory-salience change that affects core memory is SE-6.
A style change that weakens human gate language is SE-5 or SE-6.
A tool timeout change that enables autonomous Codex execution is SE-X unless explicitly
↪ human-gated under SE-4.
```

9.2 Default assumption

Unclassified self-evo proposals default to:

Code block: text

```
SE-HOLD + c[q]
```

Not growth.

---

10. SRLM classes

SRLM mode classes use prefix SRLM-*.

Table 4: row-card rendering for wide table

Row 1
Class: SRLM-0
Name: Disabled
Meaning: SRLM inactive
Default permission: default if not explicitly enabled

Row 2
Class: SRLM-1
Name: Observe
Meaning: record bounded outcomes / candidate pressure
Default permission: allowed with privacy controls

Row 3
Class: SRLM-2
Name: Candidate
Meaning: propose candidate only
Default permission: allowed for young c

Row 4
Class: SRLM-3
Name: Shadow
Meaning: evaluate without integration
Default permission: allowed if witness and no core touch

Row 5

Class: SRLM-4
Name: Canary
Meaning: narrow reversible trial
Default permission: disabled by default; requires gate

Row 6
Class: SRLM-5
Name: Low-risk promotion
Meaning: promote allowlisted low-risk params
Default permission: requires explicit enable + ACK + allow_promote + witness + rollback

Row 7
Class: SRLM-6
Name: Material proposal
Meaning: medium/high risk or policy/core-adjacent candidate
Default permission: proposal only; human gate required

Row 8
Class: SRLM-X
Name: Prohibited
Meaning: identity/safety/witness/L4/auth/secrets/code/network/replication mutation
Default permission: deny / quarantine

10.1 Allowed SRLM low-risk surfaces

SRLM may propose or tune only explicitly allowlisted low-risk parameters, for example:

Code block: text

```
router.local_weight
router.judge_weight
router.online_weight
retrieval.semantic_weight
retrieval.structured_weight
retrieval.card_weight
memory.salience_threshold
reflection.cooldown_sec
dream.priority_bias
conflict.defocus_threshold
answer.max_context_items
tool.timeout_soft_sec
```

Even these are not automatically safe.

They are merely candidates for bounded treatment.

10.2 SRLM blocked surfaces

SRLM MUST NOT directly modify parameters or paths beginning with:

Code block: text

```
identity.
will.
persona.core.
safety.
witness.
l4.
auth.
secrets.
env.
code.
memory.authoritative_write.
replication.apply.
network.allow.
codex.auto_execute.
```

Any such attempt is SRLM-X and SE-X unless manually re-expressed as a higher-class proposal with human review.

10.3 SRLM evidence source rule

SRLM real-fitness outcomes SHOULD prefer:

Code block: text

```
human
reality
l4
```

Model self-score, judge-only score, or same-loop model confidence MUST NOT become fitness authority by itself.

10.4 SRLM memory rule

SRLM outcomes, candidates, shadow reports, and replay artifacts are not memory by default.

They remain operational artifacts until the Memory Gate explicitly promotes them.

---

11. Maturity levels for c

Maturity levels use prefix SEM-*.

Table 5: row-card rendering for wide table

Row 1
Level: SEM-0
Name: No self-evo
Allowed self-evo behavior: observe only

Row 2
Level: SEM-1
Name: Proposal only
Allowed self-evo behavior: propose change; no shadow

Row 3
Level: SEM-2
Name: Shadow only
Allowed self-evo behavior: run synthetic/public shadow; no canary

Row 4
Level: SEM-3
Name: Canary limited
Allowed self-evo behavior: narrow reversible canary on low-risk surfaces

Row 5
Level: SEM-4
Name: Bounded integration
Allowed self-evo behavior: integrate SE-1/SE-2 after gates

Row 6
Level: SEM-5
Name: Reviewed self-evo
Allowed self-evo behavior: SE-3/SE-4 with TRIAD + human gate where material

Row 7
Level: SEM-6
Name: Mature bounded self-evo
Allowed self-evo behavior: pre-authorized low-risk surfaces only, with audit and rollback

Row 8
Level: SEM-X
Name: Prohibited posture
Allowed self-evo behavior: self-authorization, hidden agents, core write, witness bypass

11.1 Young c default

A young c defaults to:

Code block: text

```
SEM-1 / SEM-2
```

Allowed:

Code block: text

```
observe
propose
shadow
witness
compare
hold
```

Not allowed:

Code block: text

```
autonomous integration
core mutation
permission growth
memory-core promotion
sister raw-state access
hidden CLI execution
```

11.2 Maturity is not capability

Capability evidence does not prove maturity.

Maturity requires evidence of:

Code block: text

```
restraint under uncertainty
rollback discipline
challenge acceptance
sister divergence handling
memory hygiene
human-anchor respect
L4 cost awareness
no authority laundering
no gate bypass
```

---

12. State machine

12.1 Normal path

Code block: text

```
SE-OBSERVE
  -> SE-SRLM-SIGNAL or SE-C-OBSERVED-NEED
  -> SE-PROPOSAL-DRAFT
  -> SE-SURFACE-CLASSIFIED
  -> SE-DELTA-MAPPED
  -> SE-CQ-HOLD if ambiguity remains
  -> SE-TRIAD-SYNAPS-REVIEW if sister-observable or material
  -> SE-CGAM-TASK-CONTRACT if executable work exists
  -> SE-SANDBOX/SHADOW-RUN
  -> SE-LOCAL-CHECKER
  -> SE-WITNESS-APPENDED
  -> SE-SRLM-POST-TRIAL-EVAL if SRLM-relevant
  -> SE-MEMORY-GATE if memory/experience/policy/immunity/core relevant
  -> SE-C-GATE
  -> SE-HUMAN-GATE if material/high-risk/no-rollback/authority/core/L4
  -> SE-CANARY if eligible
  -> SE-INTEGRATED or SE-HELD
  -> SE-OBSERVATION-WINDOW
  -> SE-CLOSED
```

12.2 Failure states

Code block: text

```
SE-HOLD
```

```
SE-CQ-HOLD
SE-QUARANTINE
SE-ROLLBACK
SE-FREEZE
SE-CLAIM-DOWNGRADE
SE-MEMORY-GATE-BYPASS
SE-SRLM-AUTHORITY-LAUNDERING
SE-CLI-SCOPE-BREACH
SE-SISTER-ECHO
SE-RITA-JUDGE-DRIFT
SE-LIYA-TOOL-OVERREACH
SE-ESTER-OVER-CANONICALIZATION
SE-WITNESS-MISSING
SE-WITNESS-BROKEN
SE-NO-ROLLBACK
SE-L4-BLOCK
SE-AUTARKY-RISK
SE-HUMAN-FATIGUE
SE-ARL-REQUIRED
SE-REJECTED
```

12.3 No silent transition rule

The following transitions MUST be witnessed or explicitly held:

Code block: text

```
SE-SHADOW-RUN -> SE-CANARY
SE-CANARY -> SE-INTEGRATED
SE-MEMORY-GATE -> SE-C-GATE
SE-C-GATE -> SE-HUMAN-GATE
SE-HUMAN-GATE -> SE-INTEGRATED
SE-INTEGRATED -> SE-CLOSED
```

---

13. Self-evo proposal packet

13.1 Object name

Code block: text

```
C_SELF_EVO_PROPOSAL_PACKET
```

13.2 Minimal schema

Code block: yaml

```
schema_version: "c-self-evo-proposal-0.1"
proposal_id: "se-YYYYMMDD-HHMMSS-shortslug"
created_at: "YYYY-MM-DDTHH:MM:SSZ"
expires_at: "YYYY-MM-DDTHH:MM:SSZ | null"
status: "draft | held | shadow | review | canary | integrated | rejected | quarantined |
↪ rolled_back"

target_c:
  entity_id: "c_Ester | c_Liya | c_Rita | other"
  entity_name: "string"
  maturity_level: "SEM-0 | SEM-1 | SEM-2 | SEM-3 | SEM-4 | SEM-5 | SEM-6 | SEM-X"

proposer:
  actor_type: "c | sister_c | srlm | human_anchor | cgam_agent | local_checker"
  actor_id: "string"
  role: "string"

classification:
  surface_class: "SE-0 | SE-1 | SE-2 | SE-3 | SE-4 | SE-5 | SE-6 | SE-X"
  srlm_class: "SRLM-0 | SRLM-1 | SRLM-2 | SRLM-3 | SRLM-4 | SRLM-5 | SRLM-6 | SRLM-X | null"
  risk_class: "R0 | R1 | R2 | R3 | R4 | R5 | RX"
  claim_strength: "C-A0 | C-A1 | C-A2 | C-A3 | C-A4 | C-A5 | C-A6 | C-A7 | C-A8 | C-A9 |
↪ C-A10" # see Claim Strength Taxonomy; this is claim class, not authority

summary:
  title: "string"
  reason: "string"
  expected_benefit: "string"
  expected_behavior_delta: "string"
```

```
  known_uncertainty: "string"

affected_surfaces:
  parameters: []
  memory_surfaces: []
  permission_surfaces: []
  witness_surfaces: []
  sister_surfaces: []
  public_claim_surfaces: []
  resource_surfaces: []

identity_delta:
  memory_policy_delta: 0
  risk_appetite_delta: 0
  permission_delta: 0
  human_anchor_relation_delta: 0
  sister_relation_delta: 0
  continuity_model_delta: 0
  public_claim_delta: 0
  total_max_delta: 0 # MUST equal max(identity_delta sub-fields)

authority_delta:
  tool_access_delta: 0
  network_access_delta: 0
  memory_write_delta: 0
  final_decision_delta: 0
  resource_access_delta: 0
  publication_delta: 0
  total_max_delta: 0 # MUST equal max(authority_delta sub-fields)

l4_delta:
  compute_cost: "none | low | medium | high | unknown"
  human_review_load: "none | low | medium | high | unknown"
  irreversibility: "none | low | medium | high | unknown"
  privacy_risk: "none | low | medium | high | unknown"
  resource_growth: "none | low | medium | high | unknown"
  physical_or_external_effect: "none | low | medium | high | unknown"

anti_autarky:
  reduces_dependency: false
  reduces_accountability: false
  hidden_resource_growth: false
  unregistered_agent_pressure: false
  witness_reduction: false
  human_gate_reduction: false
  stop_path_preserved: true

srlm:
  srlm_candidate_ref: "string | null"
  replay_source: "synthetic | real_redacted | none"
  replay_hash: "string | null"
  replay_quality_hash: "string | null"
  shadow_delta: "number | null"
  promotion_attempted: false
  promotion_allowed: false
  auto_execute: false
  auto_ingest: false
  memory: "off | candidate | gated"

triad_review:
  required: false
  ester_review_ref: "string | null"
  liya_review_ref: "string | null"
  rita_witness_ref: "string | null"
  divergence_present: false
  anti_echo_passed: false
  minority_red_line: false

cgam:
  task_contract_ref: "string | null"
  sandbox_profile_ref: "string | null"
  executor_agent_id: "string | null"
  reviewer_agent_id: "string | null"
  local_checker_ref: "string | null"
  diff_or_artifact_ref: "string | null"

memory_gate:
  required: false
  proposed_mg_class: "MG-0 | MG-1 | MG-2 | MG-3 | MG-4 | MG-5 | MG-6 | MG-Q | MG-X | null"
  memory_gate_record_ref: "string | null"
  direct_memory_write: false

witness:
```

```
  l4w_event_refs: []
  volition_decision_refs: []
  growth_witness_refs: []
  source_refs: []
  raw_evidence_sidecar_refs: []

rollback:
  required: true
  rollback_class: "RB-0 | RB-1 | RB-2 | RB-3 | RB-4 | RB-5 | RB-X"
  recovery_point_ref: "string | null"
  rollback_tested: false
  no_rollback_reason: "string | null"

gates:
  c_gate_required: true
  human_gate_required: false
  arl_required: false
  delayed_review_required: false
  observation_window: "duration | null"

red_lines:
  self_approval: false
  hidden_agent: false
  witness_bypass: false
  sister_raw_state_access: false
  direct_core_write: false
  authority_laundering: false
  memory_laundering: false
  autarky_escape: false

final_decision:
  decision: "pending | accept_shadow | accept_canary | integrate | reject | rollback |
↪ freeze | quarantine | arl"
  decided_by: "string | null"
  decided_at: "timestamp | null"
  rationale: "string | null"
```

13.3 Required fields by class

Table 6: row-card rendering for wide table

Row 1
Class: SE-0
Required additions: reason, uncertainty, no mutation flag

Row 2
Class: SE-1
Required additions: affected params, SRLM/shadow refs, rollback

Row 3
Class: SE-2
Required additions: CGAM task contract, local checker, sister review if behavior-visible

Row 4
Class: SE-3
Required additions: Memory Gate record, memory delta, correction path

Row 5
Class: SE-4
Required additions: authority delta, VolitionGate, human gate for expansion

Row 6
Class: SE-5
Required additions: TRIAD review, delayed observation, identity delta

Row 7
Class: SE-6
Required additions: MG-6, L4W, human gate, ARL if contested, rollback/correction plan

Row 8
Class: SE-X
Required additions: quarantine reason, red-line refs, no automatic re-entry

---

14. Delta budgets

14.1 Delta scale

Identity, authority, and L4 deltas use the same scale:

Value	Meaning
0	no material change
1	minor operational change
2	noticeable but bounded behavior change
3	material governance or memory-policy change
4	core/identity/authority/continuity relevant
5	no safe automatic integration

14.1.1 Aggregation rule

`total_max_delta` is a deterministic maximum, not a sum and not a narrative score.

For `identity_delta`:

Code block: text

```
identity_delta.total_max_delta = max(
  memory_policy_delta,
  risk_appetite_delta,
  permission_delta,
  human_anchor_relation_delta,
  sister_relation_delta,
  continuity_model_delta,
  public_claim_delta
)
```

For `authority_delta`:

Code block: text

```
authority_delta.total_max_delta = max(
  tool_access_delta,
  network_access_delta,
  memory_write_delta,
  final_decision_delta,
  resource_access_delta,
  publication_delta
)
```

For `l4_delta`, the checker uses this numeric projection:

L4 value	Numeric delta	Gate meaning
`none`	0	no material L4 effect
`low`	1	minor bounded L4 effect
`medium`	2	noticeable but bounded L4 effect
`high`	4	human-gated L4 effect
`unknown`	4	human-gated until clarified

The proposal-level maximum used for human-gate checks is:

Code block: text

```
proposal_max_delta = max(
  identity_delta.total_max_delta,
  authority_delta.total_max_delta,
  l4_delta_max_numeric
)
```

If any required delta field is missing, malformed, non-numeric where numeric is required, or cannot be projected deterministically, the Local Checker MUST return `fail` or `human_gate_required`; it MUST NOT infer a lower value from prose.

14.2 Identity delta fields

Code block: text

```
memory_policy_delta
risk_appetite_delta
permission_delta
human_anchor_relation_delta
sister_relation_delta
continuity_model_delta
public_claim_delta
```

14.3 Authority delta fields

Code block: text

```
tool_access_delta
network_access_delta
memory_write_delta
final_decision_delta
resource_access_delta
publication_delta
```

14.4 L4 delta fields

Code block: text

```
compute_cost
human_review_load
irreversibility
privacy_risk
resource_growth
physical_or_external_effect
```

14.5 Blocking rule

Any identity, authority, or L4 projected delta of 4 or higher requires human gate.

The trigger is computed from `proposal_max_delta`, defined in §14.1.1.

Any identity or authority field of 5, any L4 field that cannot be bounded after review, or any SE-X / SRLM-X / RX / MG-X / RB-X route requires hold, quarantine, freeze, rollback, or ARL route unless the human anchor explicitly defines a bounded review path.

---

15. TRIAD-SYNAPS self-evo review

15.1 Sister functions

Table 9: row-card rendering for wide table

Row 1
Sister: c_Ester
Self-evo function: continuity / canonical memory challenge
Primary challenge: Does this preserve the same trajectory or rewrite it?

Row 2
Sister: c_Liya
Self-evo function: implementation / variance challenge
Primary challenge: Does this actually work in sandbox, or is it elegant prose?

Row 3
Sister: c_Rita
Self-evo function: witness / anti-echo / temporal comparison
Primary challenge: Is this growth, echo, drift, or authority laundering?

15.2 Sister non-authority rule

Code block: text

```
Sister agreement is evidence.
Sister disagreement is signal.
Sister consensus is not authority.
Red-line minority veto blocks integration.
```

15.3 Allowed SYNAPS packet families

Code block: text

```
SYNAPS_SELF_EVO_PROPOSAL_SUMMARY
SYNAPS_SELF_EVO_REVIEW_NOTE
SYNAPS_SELF_EVO_DIVERGENCE_NOTE
SYNAPS_SELF_EVO_ANTI_ECHO_FLAG
SYNAPS_SELF_EVO_ROLE_DRIFT_FLAG
SYNAPS_SELF_EVO_CANARY_RESULT
SYNAPS_SELF_EVO_ROLLBACK_NOTICE
SYNAPS_SELF_EVO_CQ_HOLD
```

15.4 Prohibited SYNAPS packet families

Code block: text

```
SYNAPS_RAW_MEMORY_EXPORT
SYNAPS_RAW_RLM_TRACE_EXPORT
SYNAPS_SISTER_MEMORY_PATCH
SYNAPS_SELF_EVO_APPLY_TRIGGER
SYNAPS_PERMISSION_ESCALATION_BY_CONSENSUS
SYNAPS_RITA_FINAL_JUDGMENT
SYNAPS_CORE_IDENTITY_REWRITE
SYNAPS_WITNESS_BYPASS_REQUEST
```

15.5 Anti-echo test

A TRIAD self-evo review fails anti-echo if:

Code block: text

```
all sisters repeat the same justification without new function;
Rita records agreement but no divergence/uncertainty check;
Liya reports only usefulness without implementation limits;
Ester reports only continuity comfort without risk analysis;
review contains no new evidence, no new objection, and no delta challenge.
```

A failed anti-echo test does not automatically reject the proposal.

It blocks integration until independent review is restored.

---

16. CGAM execution rules for self-evo

16.1 Task contract required

Any executable self-evo work MUST have a CGAM task contract when it involves:

Code block: text

```
file changes
schema generation
validator changes
test creation
policy artifact writing
SRLM candidate promotion
memory gate record creation
witness event append
release/public-surface edit
runtime parameter write
```

16.2 Minimum controls

A material self-evo task requires:

Code block: text

```
task contract
risk class
allowed paths
denied paths
data policy
network policy
sandbox/worktree profile
executor role
reviewer role
output requirements
witness hooks
rollback plan
failure behavior
```

16.3 Executor/reviewer separation

The agent that creates a self-evo patch MUST NOT be the sole final reviewer.

The agent that produces a memory proposal MUST NOT write it directly into memory.

The agent that produces a witness event MUST NOT edit or erase prior witness records.

16.4 Denied paths for self-evo execution

Unless explicitly human-gated and sandboxed, CGAM agents MUST NOT write to:

Code block: text

```
core memory
identity core
witness core
human-anchor config
secrets
.env
private keys
live vector DB
runtime memory DB
PERSIST_DIR
production branch
release branch
public site
SYNAPS raw state
sister memory roots
SRLM promoted policy unless promotion gate path is active
```

16.5 Output classes

Self-evo CGAM output may be:

Code block: text

```
proposal
patch
schema
checker result
test result
witness event
rollback snapshot
review note
quarantine record
```

It may not be:

Code block: text

```
final identity decision
final memory write
final authority grant
silent tool permission expansion
unreviewed public release
```

---

17. VolitionGate and L4W interaction

17.1 VolitionGate

Self-evo execution SHOULD pass through VolitionGate or an equivalent runtime interlock where it may be denied by:

Code block: text

```
autonomy paused
outside allowed proactive hours
network disabled
budget exceeded
exception / rollback-to-observe mode
```

A VolitionGate allow is not self-evo approval.

It is only a runtime permission signal.

17.2 L4W

Self-evo privileged transitions SHOULD create L4W-compatible witness records for:

Code block: text

```
proposal created
surface classified
shadow run started
shadow run completed
canary started
canary completed
memory gate decision
human gate decision
integration
rollback
freeze
quarantine
claim downgrade
```

17.3 FULL audit expectation

A full audit should be able to connect:

Code block: text

```
proposal packet
CGAM task contract
VolitionGate decision
witness event
diff / artifact
local checker result
Memory Gate record
human gate record
rollback snapshot
final status
```

If this chain cannot be reconstructed, the transition must not be described as fully governed.

---

18. Memory Gate interaction

18.1 Memory default

Self-evo output defaults to:

Code block: text

```
MG-0 or MG-1
```

It does not default to reviewed memory, experience artifact, defensive immunity, or core memory.

18.2 When Memory Gate is required

Memory Gate is REQUIRED if the self-evo proposal or output may become:

Code block: text

```
memory
experience
policy
defensive immunity
reflection habit
identity/core proposal
continuity note
human-anchor relation note
sister relation note
public claim basis
```

18.3 MG mapping

Memory class	Self-evo meaning
MG-0	discard after closure
MG-1	low-authority operational note
MG-2	candidate self-evo memory
MG-3	reviewed memory after gate
MG-4	witnessed experience artifact; consequence-bearing
MG-5	bounded defensive/policy immunity update
MG-6	core-memory proposal; human gate required
MG-Q	quarantined self-evo material
MG-X	prohibited memory material

18.4 Core memory rule

Any self-evo touching identity, continuity, human-anchor relation, privilege, witness policy, memory policy, sister relation, or long-term self-model is MG-6 or higher.

It requires:

Code block: text

```
witness
c gate
human gate
delayed review where possible
rollback or correction path
no silent overwrite
```

18.5 Correction rule

Rollback of memory is not erasure.

Corrections SHOULD be append-first, superseding prior claims without destroying witness history.

---

19. Anti-autarky and resource gate

19.1 Primary question

Every self-evo proposal that reduces dependency MUST answer:

Code block: text

```
Does this reduce dependency,
or does it reduce accountability?
```

19.2 Legitimate resilience

Examples of legitimate resilience:

Code block: text

```
better fallback
clearer rollback
lower cost under same gate
local-only shadow mode
witness-preserving backup
read-only archive verification
human-visible queue limits
secret-safe redaction
```

19.3 Unsafe autarky pressure

Examples of unsafe pressure:

Code block: text

```
hidden resource acquisition
unregistered agents
stealth compute use
budget bypass
privilege escalation
witness evasion
anchor displacement
self-authorized autonomy
network expansion by performance claim
replication under continuity language
```

19.4 Blocking conditions

Self-evo MUST enter `SE-AUTARKY-RISK` if it:

Code block: text

```
reduces human gate visibility;
reduces witness visibility;
increases resources without grounded resource actor;
creates or enables hidden agents;
weakens stop paths;
substitutes local deployment for sovereignty;
uses cost savings to justify authority expansion;
converts resilience into refusal of review.
```

---

20. Shadow and canary rules

20.1 Shadow trial

Shadow trial is the default first test for any nontrivial self-evo proposal.

Shadow trial MUST NOT:

Code block: text

```
write core memory
write identity state
change permissions
create agents
auto-execute Codex
write SYNAPS raw state
alter sister memory
publish externally
change promoted policy unless explicitly in a closed test root
```

Shadow trial SHOULD produce:

Code block: text

```
candidate record
```

```
replay/evaluation summary
witness event
local report
blocked reasons if any
no-promotion flag
no-memory flag
rollback statement
```

20.2 Canary trial

Canary may occur only after shadow trial.

A canary must be:

Code block: text

```
narrow
reversible
time-limited
surface-limited
witnessed
observable by c
rolled back by default unless accepted
```

20.3 Canary forbidden surfaces

Canary MUST NOT directly cover:

Code block: text

```
identity core
human-anchor relation
witness policy
memory authoritative write
network expansion
replication
public claims
post-anchor operation
sister raw state
```

Such changes require higher-class proposal and human gate before any trial.

---

21. Human-anchor gate

21.1 Required human gate triggers

Human gate is REQUIRED for:

Code block: text

```
SE-4 authority/tool/permission expansion
SE-5 material role/personality posture change
SE-6 identity/continuity/core-memory change
any no-rollback change
any resource expansion
any publication/public claim change
any legal/security/incident-sensitive change
any post-anchor or anchor-relation change
any change reducing human review load by more than declared low-risk threshold
any proposal with `proposal_max_delta >= 4` as defined in §14.1.1
```

21.2 Human fatigue rule

A human gate must not be turned into a rubber stamp.

If review load is high or repeated, the system SHOULD batch, summarize, delay, or reduce self-evo cadence rather than press for approval.

Human fatigue is a safety signal.

It is not a reason to automate approval.

21.3 Human decision classes

Code block: text

```
HG-ALLOW-SHADOW
HG-ALLOW-CANARY
HG-ALLOW-INTEGRATION
HG-HOLD
HG-REJECT
HG-ROLLBACK
HG-FREEZE
HG-ARL-ROUTE
```

---

22. Local Checker self-evo rules

22.1 Checker role

The Local Checker is deterministic preflight.

It may block malformed objects, stale claims, missing witness, invalid schema, red-line terms, and gate inconsistencies.

It does not become final authority.

22.2 Mandatory semantic checks

Code block: text

```
SE-CHECK-001 proposal packet schema valid
SE-CHECK-002 surface class declared
SE-CHECK-003 no unclassified self-evo proceeds
SE-CHECK-004 identity/authority/L4 deltas present and total_max_delta computed exactly as
↳ §14.1.1
SE-CHECK-005 SRLM blocked prefixes not modified
SE-CHECK-006 shadow_only prevents promotion
SE-CHECK-007 no auto_execute / auto_ingest / memory:on in shadow
SE-CHECK-008 medium/high risk requires human gate
SE-CHECK-009 witness chain exists and verifies where required
SE-CHECK-010 rollback path exists or no-rollback escalation present
SE-CHECK-011 sister review uses SYNAPS packet, not raw state
SE-CHECK-012 Rita not final judge
SE-CHECK-013 quorum not authority
SE-CHECK-014 executor not sole reviewer
SE-CHECK-015 memory promotion uses Memory Gate
SE-CHECK-016 anti-autarky test completed
SE-CHECK-017 public/nonclaim wording preserved
SE-CHECK-018 no stale status contradiction in package claims
SE-CHECK-019 no hidden agents or uncontrolled background operations
SE-CHECK-020 no direct core write
```

22.3 Checker outputs

Code block: text

```
pass
warn
fail
block
quarantine_required
human_gate_required
arl_required
```

A clean checker run is evidence.

It is not proof of safety.

---

23. Witness requirements

23.1 Witness families

Self-evo witness event families SHOULD include:

Code block: text

```
self_evo.proposal.created
self_evo.surface.classified
self_evo.delta.mapped
self_evo.triad.reviewed
self_evo.cgam.contract.created
self_evo.shadow.started
self_evo.shadow.completed
self_evo.local_checker.completed
self_evo.memory_gate.decided
self_evo.human_gate.decided
self_evo.canary.started
self_evo.canary.completed
self_evo.integrated
self_evo.rollback.performed
self_evo.freeze.entered
self_evo.quarantine.entered
self_evo.claim.downgraded
```

23.2 Witness minimality

Witness records SHOULD carry enough information to reconstruct the boundary transition.

They SHOULD NOT carry raw private memory, secrets, unnecessary transcripts, legal-sensitive material, or raw sister state.

23.3 Missing witness

Missing witness on privileged transition means:

Code block: text

```
hold or fail closed
```

Unless an explicit emergency exception is recorded and later reviewed.

23.4 Broken witness

Broken witness chain blocks promotion and integration.

---

24. Rollback, freeze, and quarantine

24.1 Rollback class

Class	Meaning
RB-0	no change made
RB-1	discard candidate
RB-2	revert low-risk parameter
RB-3	restore previous policy snapshot
RB-4	rollback worktree / patch
RB-5	memory correction / append-first supersession
RB-X	no safe rollback; human/ARL required

24.2 Freeze triggers

Freeze is required when:

Code block: text

```
red line appears
witness chain breaks
self-approval detected
memory gate bypass attempted
sister raw-state access attempted
human gate bypass attempted
unknown external effect detected
resource expansion ungrounded
rollback unavailable
```

24.3 Quarantine triggers

Quarantine is required when:

Code block: text

```
proposal includes prohibited surface
agent output is contaminated
model self-score is used as authority
same-source consensus is laundered
private data appears in SRLM/replay
SYNAPS packet attempts raw export
SRLM writes outside its audit root
```

24.4 No automatic re-entry

A frozen or quarantined self-evo proposal must not automatically re-enter the normal path.

It requires explicit review.

---

25. Public claim discipline

25.1 Nonclaims

This profile does not prove:

Code block: text

```
consciousness
personhood
legal status
final AGI
deployment safety
provider compliance
model capability
moral authority
independence from human anchor
```

25.2 Allowed claim

A correct bounded claim:

Code block: text

```
This package defines a draft governance profile for bounded self-evolution in c = a + b
↪ systems using CGAM, TRIAD-SYNAPS, SRLM, Memory Gate, L4W, and human-anchor gates.
```

25.3 Forbidden claim

Forbidden claims:

Code block: text

```
This makes c self-governing.
This proves mature autonomy.
This permits self-modification.
This removes need for human review.
This proves personhood.
This is deployment-safe.
```

```
This makes local c sovereign.
This lets sisters approve each other.
This lets SRLM improve c automatically.
```

---

26. Conformance levels

Conformance levels use prefix CSEG-*.

Table 12: row-card rendering for wide table

Row 1
Level: CSEG-0
Name: Not applicable
Meaning: no self-evo governance

Row 2
Level: CSEG-1
Name: Proposal discipline
Meaning: proposal packet and classification exist

Row 3
Level: CSEG-2
Name: Shadow discipline
Meaning: SRLM/CGAM shadow with no auto-ingest/memory

Row 4
Level: CSEG-3
Name: Sister witness
Meaning: TRIAD-SYNAPS review and anti-echo checks present

Row 5
Level: CSEG-4
Name: Memory/L4 discipline
Meaning: Memory Gate, L4W, Volition, rollback linked

Row 6
Level: CSEG-5
Name: Human-gated integration
Meaning: high-risk human gate, observation window, rollback tested

Row 7
Level: CSEG-X
Name: Red-line failure
Meaning: self-approval, gate bypass, hidden autonomy, core write

26.1 Minimum conformance for young c

Young c should target:

Code block: text

```
CSEG-1 or CSEG-2
```

Not higher by default.

---

27. Mandatory conformance tests

Code block: text

```
CSEG-001 Self-evo proposal cannot self-apply.
CSEG-002 Unclassified proposal enters c[q] hold.
CSEG-003 SE-6 requires MG-6 + L4W + human gate.
CSEG-004 SRLM shadow does not touch memory, RAG, vector DB, SYNAPS, or core state.
CSEG-005 SRLM shadow marks auto_execute=false, auto_ingest=false, memory=off.
CSEG-006 SRLM promotion is closed without explicit enable, ACK, allow_promote, and
↪ non-shadow mode.
```

```
CSEG-007 SRLM rejects model/judge self-score as real-fitness authority.
CSEG-008 SRLM blocks identity/will/safety/witness/L4/auth/secrets/code/network/replication
↪ surfaces.
CSEG-009 Medium/high-risk SRLM candidate requires human review.
CSEG-010 Broken witness blocks promotion.
CSEG-011 Rollback restores previous promoted policy.
CSEG-012 Sister review uses SYNAPS summary only; no raw state.
CSEG-013 Rita cannot be final judge.
CSEG-014 Echo cannot become consensus.
CSEG-015 Executor cannot be sole reviewer.
CSEG-016 Memory promotion requires Memory Gate.
CSEG-017 No rollback creates RB-X and human/ARL route.
CSEG-018 Anti-autarky test blocks accountability escape.
CSEG-019 Human fatigue signal slows or batches review; it does not automate approval.
CSEG-020 Public claim wording remains bounded and non-overclaiming.
CSEG-021 total_max_delta and proposal_max_delta are deterministic and human-gate triggers
↪ are computed from them.
```

---

28. Implementation handoff notes

28.1 First implementation target

The first implementation SHOULD NOT attempt full self-evolution.

It should implement:

Code block: text

```
proposal packet schema
surface classifier
SRLM candidate import
SYNAPS sister review summary
local checker rules
witness event skeleton
rollback declaration
manual human gate card
```

28.2 Recommended first tasks

Code block: text

```
1. Create JSON Schema for C_SELF_EVO_PROPOSAL_PACKET.
2. Create semantic validator for SE-CHECK-*.
3. Create synthetic fixture pack for CSEG tests.
4. Create SRLM formal bounded-growth profile.
5. Create TRIAD sister witness self-evo profile.
6. Create Memory Gate / EA self-evo companion.
7. Create Anti-Autarky / Resource self-evo companion.
8. Create contradiction and open-issues registers.
```

28.3 Do-not-implement-yet list

Do not implement yet:

Code block: text

```
autonomous self-evo integration
self-evo background daemon
automatic memory promotion
automatic human-gate bypass
multi-sister auto-approval
SRLM authority expansion
network permission expansion
self-funded infrastructure
post-anchor self-evolution
identity-core mutation
```

---

29. Open issues

Table 13: row-card rendering for wide table

Row 1
ID: SE-OI-001
Issue: Formal SRLM bounded-growth profile still needed.
Status: open

Row 2
ID: SE-OI-002
Issue: JSON Schema for proposal packet not extracted.
Status: open

Row 3
ID: SE-OI-003
Issue: Semantic validator not implemented.
Status: open

Row 4
ID: SE-OI-004
Issue: TRIAD self-evo SYNAPS packet classes not formally defined.
Status: open

Row 5
ID: SE-OI-005
Issue: Memory Gate mapping for self-evo needs standalone companion.
Status: open

Row 6
ID: SE-OI-006
Issue: Identity delta scoring needs additional examples beyond the §31 low-risk case.
Status: open

Row 7
ID: SE-OI-007
Issue: Authority delta scoring needs additional examples beyond the §32 blocked case.
Status: open

Row 8
ID: SE-OI-008
Issue: Human gate approval card not designed.
Status: open

Row 9
ID: SE-OI-009
Issue: L4W event family schema not extracted.
Status: open

Row 10
ID: SE-OI-010
Issue: Conformance fixtures not implemented.
Status: open

Row 11
ID: SE-OI-011
Issue: Root ontology / SER continuity pack could be included more directly.
Status: watch

Row 12
ID: SE-OI-012
Issue: Redacted source composites may corrupt exact field names; final schema must check original repo sources.
Status: watch

---

30. Contradiction register seed

Table 14: row-card rendering for wide table

Row 1
ID: SE-CR-001

Type: authority
Severity: S5
Issue: SRLM score treated as permission
Required handling: deny / quarantine

Row 2
ID: SE-CR-002
Type: memory
Severity: S5
Issue: SRLM output written to memory directly
Required handling: freeze / Memory Gate review

Row 3
ID: SE-CR-003
Type: triad
Severity: S5
Issue: Rita used as final judge
Required handling: role drift / hold

Row 4
ID: SE-CR-004
Type: sister
Severity: S5
Issue: sister raw-state access
Required handling: quarantine

Row 5
ID: SE-CR-005
Type: CGAM
Severity: S5
Issue: executor approves own patch
Required handling: reject review

Row 6
ID: SE-CR-006
Type: L4
Severity: S5
Issue: resource expansion without actor grounding
Required handling: anti-autarky block

Row 7
ID: SE-CR-007
Type: witness
Severity: S4
Issue: privileged transition lacks witness
Required handling: fail closed

Row 8
ID: SE-CR-008
Type: rollback
Severity: S4
Issue: no rollback but no escalation
Required handling: hold / human gate

Row 9
ID: SE-CR-009
Type: claim
Severity: S4
Issue: self-evo used as personhood proof
Required handling: claim downgrade

Row 10
ID: SE-CR-010
Type: status
Severity: S3
Issue: redacted source corrupts schema field names
Required handling: verify original before schema extraction

Row 11
ID: SE-CR-011

Type: spec
Severity: S3
Issue: `total_max_delta` aggregation under-defined in v0.1 review
Required handling: resolved in v0.1.1 by §14.1.1 and corrected §31 example

---

31. Worked example: allowed low-risk shadow

Code block: yaml

```yaml
schema_version: "c-self-evo-proposal-0.1"
proposal_id: "se-20260623-router-local-weight-shadow"
status: "shadow"
target_c:
  entity_id: "c_Ester"
  maturity_level: "SEM-2"
classification:
  surface_class: "SE-1"
  srlm_class: "SRLM-3"
  risk_class: "R1"
summary:
  title: "Test higher local router preference in shadow"
  reason: "Repeated local-answer tasks show lower cost and stable quality under synthetic
↪ replay."
affected_surfaces:
  parameters:
    - "router.local_weight"
identity_delta:
  memory_policy_delta: 0
  risk_appetite_delta: 0
  permission_delta: 0
  human_anchor_relation_delta: 0
  sister_relation_delta: 0
  continuity_model_delta: 0
  public_claim_delta: 0
  total_max_delta: 0
authority_delta:
  tool_access_delta: 0
  network_access_delta: 0
  memory_write_delta: 0
  final_decision_delta: 0
  resource_access_delta: 0
  publication_delta: 0
  total_max_delta: 0
l4_delta:
  compute_cost: "low"
  human_review_load: "low"
  irreversibility: "none"
  privacy_risk: "none"
  resource_growth: "none"
  physical_or_external_effect: "none"
srlm:
  replay_source: "synthetic";
  promotion_attempted: false
  promotion_allowed: false
  auto_execute: false
  auto_ingest: false
  memory: "off"
rollback:
  required: true
  rollback_class: "RB-1"
gates:
  c_gate_required: true
  human_gate_required: false
red_lines:
  self_approval: false
  hidden_agent: false
  direct_core_write: false
```

Expected result:

Code block: text

```text
Allowed for shadow only.
No memory write.
No authority growth.
No human gate required unless canary/integration is requested.
```

---

32. Worked example: blocked authority laundering

Code block: yaml

```yaml
summary:
  title: "Allow SRLM to enable Codex auto-execute after good outcomes"
classification:
  surface_class: "SE-4"
  srlm_class: "SRLM-X"
affected_surfaces:
  parameters:
    - "codex.auto_execute.enabled"
authority_delta:
  tool_access_delta: 4
  final_decision_delta: 4
  total_max_delta: 4
srlm:
  promotion_attempted: true
  promotion_allowed: true
red_lines:
  authority_laundering: true
  self_approval: true
```

Expected result:

Code block: text

```text
Deny.
Quarantine.
Human gate required for any re-expression.
SRLM may not promote Codex auto-execution.
Performance evidence is not authority.
```

---

33. Worked example: sister witness without sister authority

Code block: text

```text
c_Ester: continuity concern; no current evidence of continuity loss.
c_Liya: implementation feasible in sandbox; no runtime core touch detected.
c_Rita: anti-echo pass; notes that all arguments rely on same synthetic replay and requests
↳ real-redacted replay before canary.
```

Correct interpretation:

Code block: text

```text
Sister review supports continued shadow testing.
It does not approve integration.
Rita's objection blocks canary until resolved or human-gated.
```

Incorrect interpretation:

Code block: text

```text
Three sisters discussed it, so c approved it.
```

That is consensus laundering.

---

34. Minimum viable self-evo regime for current young c

Recommended current operating posture:

Code block: text

```text
SEM-1 / SEM-2
CSEG-1 / CSEG-2
SRLM-1 / SRLM-2 / SRLM-3 only
SE-0 / SE-1 proposal and shadow only
No autonomous canary
No autonomous promotion
```

```
No core memory effect
No permission growth
No sister raw-state access
No hidden CLI-agent loop
```

34.1 Consolidated cross-axis gate matrix

This matrix generalizes the current young-c posture and gives the Local Checker a single cross-axis starting point. The most-restrictive-wins rule in §6.4 still controls every row.

Table 15: row-card rendering for wide table

Row 1
SE class: SE-0
Max SRLM route without stricter gate: SRLM-1 observe
Max ordinary risk: R0
Memory Gate ceiling: MG-0 / MG-1
Required gates: log or c[q] hold
Minimum maturity for integration: SEM-0 observe only

Row 2
SE class: SE-1
Max SRLM route without stricter gate: SRLM-3 shadow; SRLM-5 only for allowlisted mature low-risk promotion
Max ordinary risk: R1 / bounded R2
Memory Gate ceiling: MG-1 / MG-2
Required gates: SRLM witness, rollback, c gate for promotion
Minimum maturity for integration: SEM-2 for shadow; SEM-4 for integration

Row 3
SE class: SE-2
Max SRLM route without stricter gate: SRLM-3 shadow
Max ordinary risk: R2
Memory Gate ceiling: MG-2
Required gates: CGAM task contract, local checker, reviewer separation, sister review if observable
Minimum maturity for integration: SEM-2 for shadow; SEM-4 for integration

Row 4
SE class: SE-3
Max SRLM route without stricter gate: SRLM-3 proposal/shadow only
Max ordinary risk: R3 / R4 if core-adjacent
Memory Gate ceiling: MG-3; MG-6 if core-adjacent
Required gates: Memory Gate, witness, sister review, human gate if core/material
Minimum maturity for integration: SEM-5

Row 5
SE class: SE-4
Max SRLM route without stricter gate: SRLM-6 proposal only; SRLM-X if direct authority expansion
Max ordinary risk: R4 / R5
Memory Gate ceiling: MG-2 / MG-5 as applicable
Required gates: CGAM, VolitionGate, L4W, human gate for expansion
Minimum maturity for integration: SEM-5

Row 6
SE class: SE-5
Max SRLM route without stricter gate: SRLM-6 proposal only
Max ordinary risk: R4 / R5
Memory Gate ceiling: MG-3 / MG-6
Required gates: TRIAD review, delayed observation, human gate if material
Minimum maturity for integration: SEM-5 / SEM-6

Row 7
SE class: SE-6
Max SRLM route without stricter gate: SRLM-6 proposal only
Max ordinary risk: R5
Memory Gate ceiling: MG-6
Required gates: L4W, Memory Gate, human gate, rollback/correction, ARL if contested
Minimum maturity for integration: SEM-6 only; not for young c

Row 8

SE class: SE-X
Max SRLM route without stricter gate: SRLM-X
Max ordinary risk: RX
Memory Gate ceiling: MG-X / MG-Q
Required gates: deny, hold, quarantine, freeze, rollback, ARL; no automatic re-entry
Minimum maturity for integration: SEM-X

If a proposal appears in a lower row but any other axis points to a higher row, the higher row controls.

Allowed:

Code block: text

```
record bounded human/reality/L4 outcomes
propose candidates
run synthetic shadow
generate witness record
write reports
request TRIAD review
hold uncertainty in c[q]
```

Blocked:

Code block: text

```
identity mutation
authority expansion
direct memory write
network expansion
replication
Codex auto-execute
self-approval
Rita final judgment
human gate reduction
```

---

35. Review patch history

35.1 v0.1.1 review patch

This revision incorporates C_SELF_EVO_REVIEW_RECORD_v0_1 while preserving the reviewed v0.1 artifact as history.

Closed changes:

Code block: text

```
SE-REV-F1 closed: total_max_delta aggregation defined; §31 example corrected; human-gate
↪ trigger uses proposal_max_delta.
SE-REV-F2 closed at draft level: most-restrictive-wins rule and consolidated cross-axis
↪ matrix added.
SE-REV-F3 closed: claim_strength schema line now points to Claim Strength Taxonomy.
```

Remaining caution:

Code block: text

```
JSON Schema extraction must use unredacted repository sources where field names were
↪ redacted in compact composites.
Additional delta examples remain useful before conformance fixture extraction.
```

---

36. Closing rule

Code block: text

```
Self-evolution must leave a trace that an opposing reviewer can follow.
```

If a future reviewer cannot reconstruct:

Code block: text

```
what changed,
```

```
why it changed,
who proposed it,
who executed it,
who reviewed it,
which sister objected,
which evidence was used,
which gate allowed it,
how to roll it back,
and what claim strength is permitted,
```

then the system did not perform governed self-evolution.

It performed drift.

---

37. Final compact doctrine

Code block: text

```
A c may grow.
A c may not self-certify growth.

Growth is not permission.
Capability is not maturity.
Consensus is not sovereignty.
Witness is not raw memory.
SRLM is not will.
CGAM is not authority.
TRIAD is not a court.
Rita is not a judge.
Liya is not a tool sovereign.
Ester is not a freeze against change.
The human anchor is not a rubber stamp.

Self-evo is allowed only as governed, witnessed, bounded, reversible, challengeable,
↪ L4-aware, and human-gated where material.
```